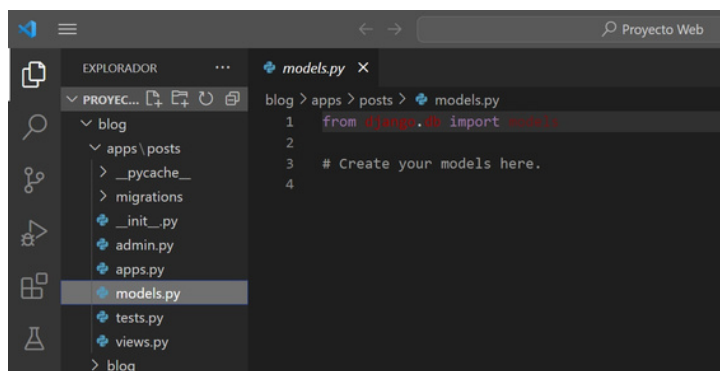


CREANDO NUESTRO PRIMER MODELO



>>> PASOS

Como pudimos adelantar en la primer parte del apunte, ahora vamos a cargar el modelo para nuestra aplicación "posts" y para eso tienes que abrir el archivo **models.py** que se encuentra en la carpeta "apps/posts" :



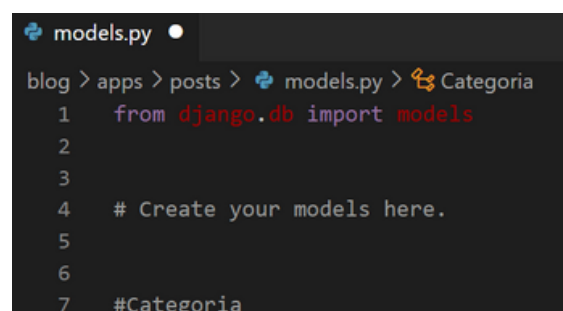
Te recordamos que los pasos que estamos siguiendo aquí (para todo) no son únicos, pueden haber diversas formas de crear un proyecto, apps, base de datos, pero siguiendo esta forma vas a poder crear tu modelo para el proyecto final sin problemas. Lo único que no va a cambiar, por más que la estructuración del proyecto no sea idéntica a esta, es la sintaxis que se requiere utilizar, por lo demás, puedes realizar variaciones.

Ahora si, manos a la obra:

- Vamos a arrancar en la línea de código 7 para dejar espacios y ser más organizados, donde primero



que nada vamos a hacer un comentario para mencionar que vamos a realizar la clase "Categoria":



Ahora crearemos una clase "Categoria" como lo hacíamos en POO, tomando a categoría como un objeto, y si lo vieramos desde SQL lo tomaríamos como una entidad.

Esta clase "Categoria" y, el resto de clases que crearemos para nuestros modelos, van a tener herencia de otra clase proporcionada por un módulo de Django - **django.db.models** - el cual se importa al iniciar (en la línea de código 1).

Al realizar esta herencia, estás indicando que esa clase es un modelo de datos que se puede almacenar y recuperar de una

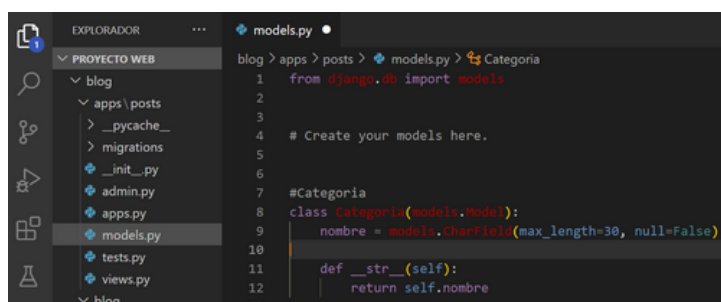
CREANDO NUESTRO PRIMER MODELO



>>> PASOS

base de datos utilizando la capa de abstracción de base de datos de Django.

Esto nos proporciona una serie de funcionalidades que nos brindan la POO y las bases de datos, como la definición de campos, métodos y atributos, relaciones entre modelos, abstracción de la base de datos ya que se obtiene acceso a una serie de características y funcionalidades útiles para trabajar con los modelos y la base de datos.



Puedes observar la herencia de la que estuvimos hablándote y, el formato que tiene la creación de la clase (como lo vimos en POO). En esta clase definimos el atributo "nombre" con un tipo de campo de tipo CharField que almacena strings con una longitud limitada (30 en este caso, ya que no se requeriría más de eso para un nombre de una categoría). Esto puede recordarte a como definíamos los campos en SQL, y si, todo está relacionado, por eso es que en este proyecto final estamos integrando todos los conocimientos adquiridos.



Además, separado por una coma, estamos indicando que el campo no puede ser nulo, por lo que se debe completar siempre con un nombre para la categoría.

Luego, más abajo, definimos el método "str" para cuando lo visualicemos, nos brinde el nombre de la categoría. Sin esta definición, cuando incluyamos una categoría, no nos mostraría el nombre, sino que aparecería como un "object" (te lo vamos a mostrar a modo de ejemplo más adelante).

Ahora nos tocaría definir la clase "Post" como tal, pero te contamos que definimos la clase "Categoría" primero para poder hacer una relación luego, entre las categorías y los posts. Ya que si no tenemos categorías, los posts no se podrían filtrar por categorías y, además, cada post quedaría desordenado. Si bien, se pueden aplicar filtros como "fecha de menor a mayor" por ejemplo, ya vas a notar lo necesario de definir una categoría para un post más adelante, y, sobre todo para este proyecto. Antes de definir el modelo, importamos un módulo que vamos a utilizar:

CREANDO NUESTRO PRIMER MODELO



>>> PASOS

```
1 from django.db import models
2 from django.utils import timezone
```

El módulo "timezone" lo vamos a utilizar para que por cada post, nos guarde la hs actual de la creación.

Definimos el modelo y te explicamos las partes:

```
14 class Post(models.Model):
15     titulo = models.CharField(max_length=50, null=False)
16     subtítulo = models.CharField(max_length=100, null=True, blank=True)
17     fecha = models.DateTimeField(auto_now_add=True)
18     texto = models.TextField(null=False)
19     activo = models.BooleanField(default=True)
20     categoria = models.ForeignKey(Categoria, on_delete=models.SET_NULL, null=True, default='Sin categoría')
21     imagen = models.ImageField(null=True, blank=True, upload_to='media', default='static/post_default.png')
22     publicado = models.DateTimeField(default=timezone.now)
```

La primer parte ya te la explicamos con "Categoria" por lo que vamos a los atributos:

- **título:** va a ser de tipo CharField con una longitud de 50 (fuimos generosos), y además no puede ser un campo nulo ya que es el título del post.
- **subtítulo:** vendría a ser como un breve resumen que se puede escribir o no, por lo que "null" y "blank" los dejamos en "True", para que no sea estrictamente necesario que se complete.
- **fecha:** usamos el tipo DateTimeField diciéndole que tome la fecha y hora actual automáticamente cuando se cree un post.
- **texto:** para este campo que va a tener el contenido del post necesitamos no limitar la cantidad de

strings que va contener, por lo que vamos a usar un tipo TextField, el cual no va a poder ser nulo y se podrá explayar todo lo necesario que requiera el contenido del post.

- **activo:** será para solo marcar una casilla (lo veremos más adelante), en la cual se tildará si un post está activo o no. Por defecto siempre que se crea un post estará activo (es decir, se publicará), sin embargo, este campo es realmente útil cuando solo queremos dejar de mostrar un post por un cierto tiempo sin tener que eliminarlo. Como puedes ver usamos un campo BooleanField que indica que este será un registro booleano.

CREANDO NUESTRO PRIMER MODELO



>>> PASOS

- **categoria:** será nuestra clave foránea a la clase "Categoria", con lo cual podremos relacionar una categoria con un post, por ende para este campo usaremos el tipo ForeignKey indicando que la relación entre ambas clases (Categoria, Post). También hacemos una salvedad, en el parámetro on_delete. Lo que estamos haciendo ahí es indicar qué sucede cuando se elimina el objeto relacionado. En este caso, se establece en SET_NULL, lo que significa que si el objeto relacionado se elimina (es decir, si se elimina una Categoria), el campo "categoria" se establecerá como NULL (vacío). Además, no requerimos que un post se encierre si o si en una categoría al momento de crearlo, ya que puede pasar que sea crea un post y no hay una categoría específica para el tipo de post y no se la quiere crear en el momento, por lo que el campo "categoria" puede quedar vacío, sin embargo, lo que realmente vamos a hacer es incluirlo en una categoría "Sin categoría" por defecto. Esto nos facilitará luego, la recategorización de los post que queden sin categoría.
- **imagen:** este campo será de la parte dinámica de la web, por lo que si aún no te quedaba claro que significaba la carpeta media y la carpeta static, ahora podrás entenderlo mejor.

Las imagenes que pueda tener el post, se guardarán mediante este campo gracias al tipo



ImageField que está preparado para aceptar archivos de tipo imagen. Este campo como puedes ver, tiene parámetros que indican que puede quedar sin completarse (null y blank = True), sin embargo, si es que ocurre esto, pondremos una imagen que estará por defecto (default='static/post_default.png') y así nos aseguraremos que todo post contenga una imagen. Las imagenes que se carguen cuando se cree un post, se subirán a la carpeta "media" ya que éstas imagenes forman la parte dinámica de la web, donde el o los usuarios que realicen la carga de post, irán subiendo o modificando las imagenes conforme se requiera, pero para las imagenes que permanecieran sin modificarse estarán en la carpeta "static" y, con esto nos referimos archivos como el logo de la web, el archivo de imagen para un post que no tendría imagen o un archivo de imagen para un usuario que se registra y no pone una imagen de perfil, entre otros archivos más que estarán indicados como fijos en la web (esto no quiere decir que los archivos

CREANDO NUESTRO PRIMER MODELO



>>> PASOS

dentro de la carpeta "static" no pueden ser modificados, solo quiere decir que van a ser archivos que en primera instancia, permanecerán ahí sin ser modificados o cambiados).

- **publicado:** aquí también utilizamos un tipo DateTimeField como en el campo "fecha", pero cambia el parámetro que le indicamos aquí. Te mostraremos mejor la diferencia entre el parámetro de "fecha" y este parámetro cuando hagamos una publicación.

Ahora definiremos una clase interna para nuestro modelo mediante la clase "Meta":

```
14 class Post(models.Model):
15     titulo = models.CharField(max_length=50, null=False)
16     subtítulo = models.CharField(max_length=100, null=True, blank=True)
17     fecha = models.DateTimeField(auto_now_add=True)
18     texto = models.TextField(null=False)
19     activo = models.BooleanField(default=True)
20     categoria = models.ForeignKey('Categoria', on_delete=models.SET_NULL, null=True, default='Sin categoria')
21     imagen = models.ImageField(null=True, blank=True, upload_to='media', default='static/post_default.png')
22     publicado = models.DateTimeField(default=datetime.now)
23
24     class Meta:
25         ordering = ('-publicado',)
```

```
23
24     class Meta:
25         ordering = ('-publicado',)
```

Meta es una clase interna dentro de la definición del modelo que define los metadatos para ese modelo en particular.



Se utiliza para definir metadatos adicionales sobre un modelo. Uno de los metadatos más comunes que se puede definir en la clase "Meta" es "ordering", que especifica el orden en que los objetos del modelo deben ser consultados o recuperados de la base de datos.

Además, como hicimos con la clase "Categoria", definiremos el método "__str__" y un método más que nos servirá para que cuando eliminemos un post, también se eliminen las imágenes asociadas a ese post. De esta forma no saturaremos el servidor con imágenes de post que ya no existen.

Este método se definirá con "delete" que realizará dos acciones adicionales antes de eliminar el post. Primero, elimina el archivo asociado al campo imagen y luego realiza la eliminación estándar del objeto llamando al método delete() de la clase padre. Esto permite añadir un comportamiento personalizado al proceso de eliminación del objeto.

CREANDO NUESTRO PRIMER MODELO



>>> PASOS

```

24 class Meta:
25     ordering = ('-publicado',)
26
27     def __str__(self):
28         return self.titulo
29
30     def delete(self, using = None, keep_parents = False):
31         self.imagen.delete(self.imagen.name)
32         super().delete()

```

Adicional al método "delete" usaremos los parámetros opcionales "using" y "keep_parents" para controlar el comportamiento específico durante la eliminación del objeto del modelo.

- **using = None:** El parámetro using se utiliza para especificar la base de datos en la cual se realizará la eliminación del objeto. Por defecto, se establece en None, lo que indica que se utilizará la base de datos predeterminada definida en la configuración de Django. Si deseas eliminar el objeto de una base de datos específica distinta a la predeterminada, puedes pasar el nombre de la base de datos como argumento al llamar al método delete().
- **keep_parents = False:** El parámetro keep_parents se utiliza en situaciones donde el modelo tiene herencia de tablas (herencia de modelos). Si se establece en True, se mantendrán los registros en las tablas de los modelos padre después de eliminar el objeto actual. Sin embargo, en este caso específico, se establece en False, lo que significa que los registros de los modelos padre no se



mantendrán y se eliminarán junto con el objeto actual.

Una vez cargado nuestro modelo y guardado (ctrl + s), iremos al siguiente paso.

Como usaremos imágenes en nuestro modelo, necesitamos instalar una dependencia más aparte de las que tenemos instaladas que son "virtualenv" y "django". Esta nueva dependencia será "pillow" quien se encargará de gestionar nuestras imágenes.

A medida que sigamos avanzando vamos a requerir instalar más dependencias, por lo que es un buen momento para crear un archivo fundamental que llevará el registro de las dependencias que estamos utilizando en nuestro proyecto. Este es el archivo "requirements.txt".

Este es un archivo de texto donde iremos anotando las dependencias con las que trabajamos en nuestro proyecto, de manera tal que, si necesitamos mudarnos de pc, por ejemplo, o si subimos nuestro

INSTALANDO DEPENDENCIAS ADICIONALES - REQUIREMENTS.TXT



»»» PASOS

repositorio a Github y alguien quisiera descargarlo y trabajar con él, con solo un comando podría instalar todas las dependencias usadas en el proyecto.

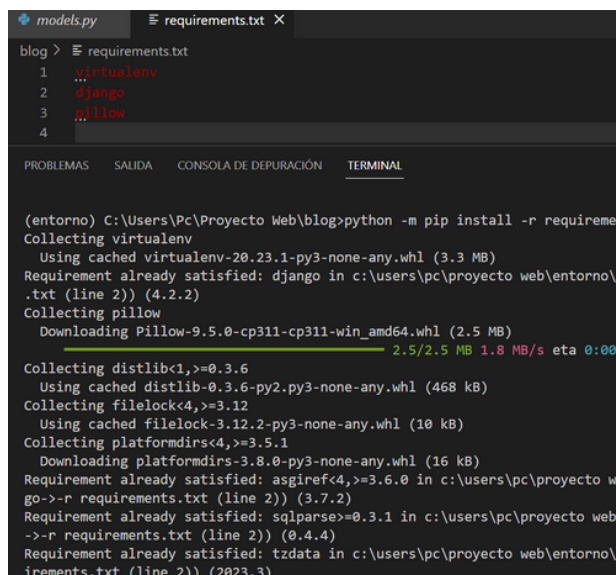
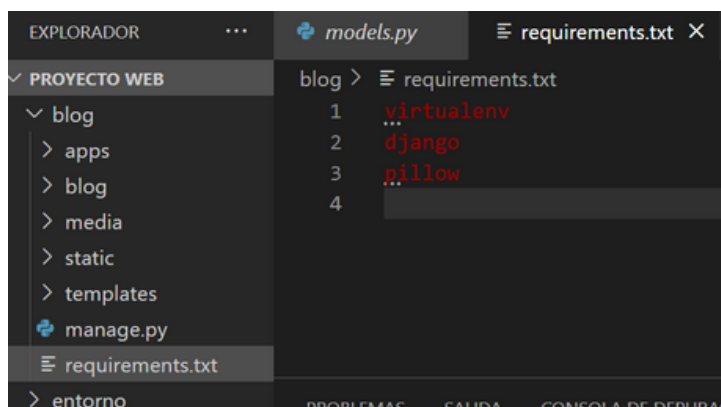
Este archivo lo podremos crear de forma manual o automática. Ahora lo haremos de forma manual, pero luego te daremos el comando para hacerlo de forma automática.

Vamos a posicionarnos en la carpeta "blog", donde se encuentra el archivo "manage.py", ahora crearemos un archivo llamado "requirements.txt" (puedes hacerlo desde VSC, desde el explorador de Windows o desde el cmd).

Una vez creado este archivo vamos a escribir las dependencias que tenemos instaladas y la que vamos a instalar, la cual, como mencionamos antes, será "pillow" y guardamos el archivo (ctrl + s).



Ahora, en la consola ejecutaremos el comando "python -m pip install -r requirements.txt".



Puedes observar que se vuelve a instalar la dependencia "virtualenv", "django" y ahora "pillow".

Este archivo "requirements.txt", puede ser instalado al principio del proyecto, no necesariamente tiene que hacerse en este

INSTALANDO DEPENDENCIAS ADICIONALES - REQUIREMENTS.TXT



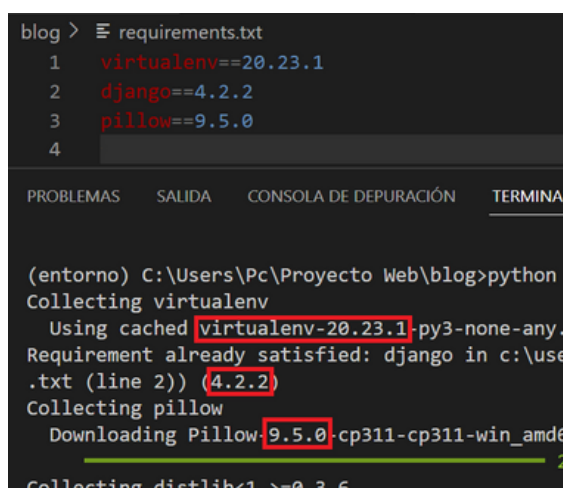
»»» PASOS

punto, pero lo hicimos de esta manera para seguir una continuidad.

Hacemos hincapié en que la estructuración del proyecto y los pasos seguidos hasta aquí no necesariamente, tienen que ser igual para cada proyecto, hay convenciones, pero no necesariamente tiene que ser así, por lo que ahora lo hicimos de esta forma para que puedas visualizar de una vez, las versiones que tenemos instaladas de nuestras dependencias, la cuales, vamos a dejarlas anotadas en nuestro archivo "requirements.txt" ya que si por algún motivo, en una nueva actualización de estas dependencias quitan o agregan algo, nuestro proyecto puede llegar a tener errores o incluso no funcionar. Además, anotamos la versión para no usar versiones anteriores o posteriores, ya que también pueden ocasionar inconvenientes en nuestro proyecto (generalmente, irregularidades en el funcionamiento).

Ahora vamos a mirar cada versión de éstas 3 dependencias que tenemos instaladas y las dejaremos anotadas en nuestro archivo.

Para dejar correctamente la versión de cada dependencia usamos la asignación con doble signo igual (==) seguida de la versión que estamos utilizando actualmente.



Con nuestro archivo "requirements.txt" completo, hasta este momento, ya podemos realizar nuestras migraciones para poder crear las nuevas tablas del modelo que acabamos de cargar.

Lo último que queda, antes de realizar las migraciones, es mostrarte el comando para que el archivo "requirements.txt", se cree de forma automática.

INSTALANDO DEPENDENCIAS ADICIONALES - REQUIREMENTS.TXT



>>> PASOS

Para realizarlo, debemos ejecutar en la consola parte de un comando que aún no te mostramos y es un buen momento de hacerlo.

Este comando lo usamos para verificar todas las dependencias que tenemos instaladas. Este comando en particular es **"pip freeze"** y, como te adelantamos nos muestra las dependencias instaladas en nuestro entorno:

```
blog > requirements.txt
1  virtualenv==20.23.1
2  django==4.2.2
3  pillow==9.5.0

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL

(entorno) C:\Users\Pc\Proyecto Web\blog>pip freeze
asgiref==3.7.2
distlib==0.3.6
Django==4.2.2
filelock==3.12.2
Pillow==9.5.0
platformdirs==3.8.0
sqlparse==0.4.4
tzdata==2023.3
virtualenv==20.23.1

(entorno) C:\Users\Pc\Proyecto Web\blog>
```



Como puede observar, este comando nos arroja las dependencias instaladas en nuestro entorno y sus versiones. Te darás cuenta que no solo están las dependencias que instalamos manualmente, sino que hay otras más. Las otras dependencias que no instalamos manualmente, fueron instaladas automáticamente con las dependencias que nosotros instalamos.

Para hacer la creación automática del archivo "requirements.txt" vamos a utilizar el comando:

- **pip freeze > requirements.txt**

Se ejecutas este comando verás que el archivo "requirements.txt" generado de forma automática, instala todas las dependencias que existen en nuestro entorno ahora, las que fueron de instalación manual, como las que fueron de instalación automática, por lo que al finalizar el proyecto y antes de subir el repositorio final a Github, te recomendamos ejecutar este comando nuevamente para que nada se pase por alto.

EJECUTANDO LAS MIGRACIONES PARA NUESTRO MODELO



»»» PASOS

Ahora es momento de generar las migraciones de nuestro modelo creado y para ello vamos a seguir los pasos que realizamos anteriormente.

Ejecutaremos el comando `python manage.py makemigrations` y el comando `python manage.py migrate`

```

PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL

(entorno) C:\Users\Pc\Proyecto Web\blog>python manage.py makemigrations
No changes detected

(entorno) C:\Users\Pc\Proyecto Web\blog>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, posts, sessions
Running migrations:
  No migrations to apply.

(entorno) C:\Users\Pc\Proyecto Web\blog>
    
```

Si bien nos aparece el mensaje "No changes detected" al realizar "makemigrations", lo hacemos, como te mencionamos antes, por una cuestión de usar estos comandos juntos. Aún sale este mensaje porque si bien, generamos un nuevo modelo, no realizamos ningún cambio sobre él, sin embargo con "migrate" si se han creado las nuevas tablas en nuestra base de datos.

Cuando comencemos a realizar ciertos cambios en nuestras tablas, "migrations" mostrará el mensaje correspondiente.

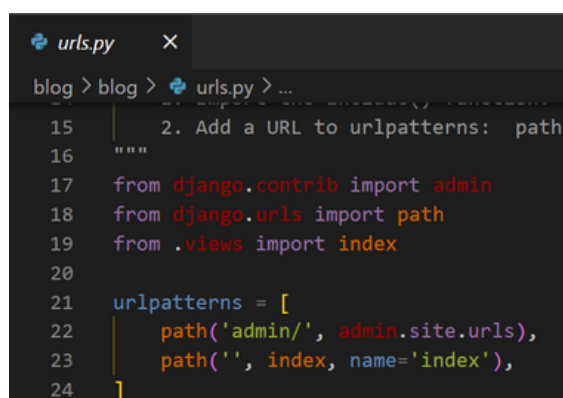
Como puede observar, al ejecutar "migrate" se ha



creado la tabla "Post" y esto es por la carga del modelo que hicimos.

Para poder ver nuestro modelo ya funcionando y poder comenzar a hacer algunas cargas de datos de prueba, vamos a hacer uso de una herramienta fundamental que nos brinda Django y es su administrador.

Si miras en el archivo "urls.py" (donde cargamos nuestra página de inicio), recordarás que pusimos nuestra url debajo de otra ya existente, y esa url es la que nos dirige al administrador que nos ofrece Django.

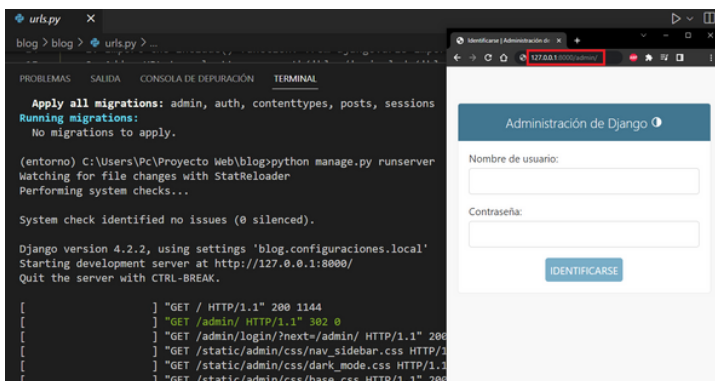


ACCEDIENDO AL ADMINISTRADOR DE DJANGO



>>> PASOS

Pues bien, si de momento quisieramos acceder a esta página, lo podríamos hacer, pero no tendríamos acceso al uso de ella, ya que nos requiere un usuario y contraseña. Ejecutamos nuestro servidor y lo probamos:



Para acceder solo debemos poner en la barra de direcciones, luego del localhost o la dirección ip del servidor "/admin" e ingresarás a la página (<http://127.0.0.1:8000/admin/>).

Sin embargo, por más que intentes poner cualquier usuario y contraseña, no podrás acceder, y esto es porque aún no hemos generado nuestro usuario y contraseña para el administrador de Django.

Para hacerlo, vamos a detener el servidor y ejecutaremos el comando:

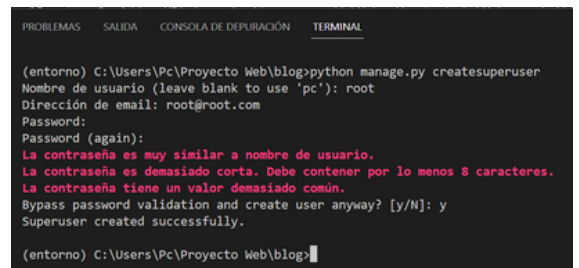
- `python manage.py createsuperuser`



Inmediatamente nos aparecerá un mensaje en el que nos pedirá que ingresemos un nombre de usuario:

```
(entorno) C:\Users\Pc\Proyecto Web\blog>python manage.py createsuperuser
Nombre de usuario (leave blank to use 'pc'):
```

Como estamos trabajando en un entorno local, y por una convención usaremos el nombre "root" y lo mismo será para los siguientes mensajes de contraseña y repetir contraseña (usaremos "root" para todo), y por último, le diremos que "si (yes - y)" cuando nos pregunte si estamos seguros (debido a lo insegura de la contraseña).



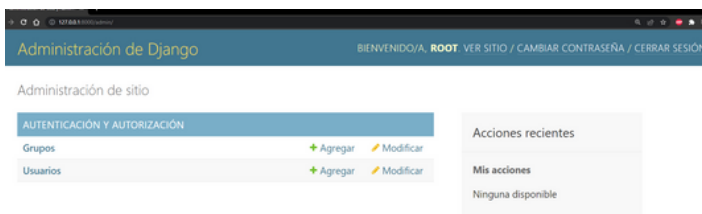
notarás que en email pusimos un mail que no existe, solo es de prueba, sin embargo, es requerido que se complete con formato de email (algo@algo.com)

ACEDIENDO AL ADMINISTRADOR DE DJANGO



>>> PASOS

Hecho esto y con el mensaje que nos dice Django que el Superusuario ha sido creado, ejecutamos el servidor y recargamos la página "admin" donde quedamos. Ahí completaremos con "root" para usuario y contraseña e ingresaremos.

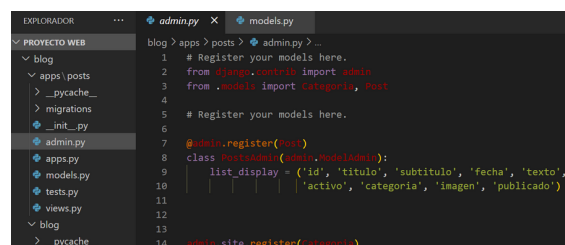


Dentro del sitio tendremos este menú, donde explicaremos cada cosa en la siguiente división del material.

Ahora, lo que podemos observar es que el modelo que cargamos "Post" no aparece y eso es porque nos falta un paso más para que pueda aparecer aquí, y este paso lo tendremos que hacer con todas las aplicaciones que crearemos más adelante, para que puedan visualizarse desde aquí.

Vamos a dirigirnos al archivo admin.py ubicado en la carpeta apps.

Dentro de este archivo, lo completaremos de la siguiente forma (no olvides que siempre debes guardar los cambios con ctrl + s):



Te mostramos las líneas a completar con zoom:

```
1 # Register your models here.
2 from django.contrib import admin
3 from .models import Categoria, Post
4
5
6
7 @admin.register(Post)
8 class PostAdmin(admin.ModelAdmin):
9     list_display = ('id', 'titulo', 'subtitulo', 'fecha', 'texto',
10                    'activo', 'categoria', 'imagen', 'publicado')
11
12
13
14 admin.site.register(Categoria)
```

Vamos a explicar las partes:

- En las líneas de código 2 y 3 estamos realizando las importaciones necesarias para que funcione todo correctamente. Por un lado hacemos una importación propia del admin de Django y por otro lado, traemos desde el archivo models.py las clases que creamos.

ACCEDIENDO AL ADMINISTRADOR DE DJANGO



>>> PASOS

- Luego en la línea de código 7 usamos la sintaxis para hacer el registro de la clase "Post".
- En la línea 8 declararemos una clase propia de Django.
- Para luego poder usar un "list_display" con los atributos que queramos mostrar (enseguida lo vas a ver) propios de la clase "Post".
- Por último, en la línea 13, hacemos un registro simple de la clase "Categoría".

Hecho esto, salvamos (ctrl + s) y nos dirigimos al navegador, donde actualizamos la página.



Y ahora si ya podemos ver nuestras clases del modelo que creamos.

Este sistema administrador que nos brinda Django, es una ayuda inmensa ya que podemos gestionar la carga de nuestra base de datos con una interfaz gráfica



funcional. Esto nos facilita mucho ya que es mucho más intuitivo que hacerlo mediante código en nuestra base de datos, nos ahorra tiempo ya que no tuvimos que crear esta interfaz, es decir, no tuvimos que crear un template para comenzar a gestionar, con lo cual, ya podemos comenzar a hacer pruebas e ir corrigiendo algunas cosas que aún no están afinadas. Incluso, hay dependencias que permiten modificar todo este sitio, pudiéndolo dejar de forma muy personalizada y con esto incluso, no es tan necesario crear un sitio completo para la administración de los datos para el usuario o cliente final.

Hasta aquí llegaremos con esta parte del apunte. En la próxima parte comenzaremos a hacer pruebas y cambiaremos los templates.